

RAPPORT DU PROJET LCAD

*StopPhishingDotCom – gestion des URLs
malintentionnés*

AIHI Imane
ISFA Lyon1

Contents

1.	Introduction	4
2.	Présentation du contexte et de l'application	4
3.	Choix et justification des technologies	5
3.1.	Backend : FastAPI + SQLAlchemy	5
3.2.	Base de données : PostgreSQL + SQLite	5
	PostgreSQL (environnement de production)	5
	SQLite (développement local et tests)	6
3.3.	Gestion du schéma : Alembic	6
3.4.	Frontend : Vue 3 + Vite	7
3.5.	Cache : Redis	7
3.6.	Conteneurisation : Docker + docker-compose	8
3.7.	Sécurité	8
3.8.	Autres outils	9
4.	Mini-documentation de l'application	10
4.1.	Architecture générale	10
4.2.	Diagramme de classes	11
4.3.	Mise en route (Docker)	12
4.4.	Implémentation :	13
	Page de connexion (Login)	13
	Page Dashboard	14
	Page /urls	15
4.5.	Déploiement :	15
4.6.	Tests	16
5.	Analyse de performance	18
5.1.	Performance générale de l'application	18
5.2.	Résultats des tests de charge (K6)	18
5.3.	Performance de l'import CSV	18
5.4.	Analyse de performance — Monitoring AWS	19
1.	CPU Utilization	19
	Network Packets Out	20
	NetworkIn & NetworkOut (volume de données)	21
6.	Conclusion et difficultés rencontrées	22

1. Introduction

Ce rapport présente **StopPhishingDotCom**, une application développée pour importer, analyser et suivre de grandes quantités d'URL afin de détecter des comportements potentiellement malveillants. L'objectif de ce document est de fournir une vision claire et structurée du projet : son rôle, son architecture, les fichiers essentiels, la procédure de mise en route, ainsi que les bonnes pratiques liées à son utilisation et à son déploiement.

2. Présentation du contexte et de l'application

Le projet consiste à concevoir une plateforme web capable de traiter en masse des fichiers CSV contenant des URL associées à différentes catégories de menace (phishing, malware, spam, etc.). L'ambition est de mettre à disposition des analystes une interface simple et efficace pour **ingérer, nettoyer, structurer, analyser et exporter** ces données.

L'application s'appuie sur une architecture moderne combinant un **backend FastAPI** et un **frontend Vue.js (Vite)**. Elle offre notamment les fonctionnalités suivantes :

- Import de fichiers CSV contenant des listes d'URL,
- Normalisation et nettoyage automatique des données (extraction du domaine, uniformisation des labels...),
- Déduplication avant insertion en base,
- Stockage structuré dans une base **PostgreSQL**,
- Visualisation des résultats via une table interactive et un tableau de bord,
- Export et suppression sélective des entrées.

L'ensemble repose sur une séparation nette des responsabilités :

- **Routes API** pour l'exposition des endpoints,
- **Services** pour la logique métier,
- **Modules utilitaires** (cache, logs, parsing CSV) pour les fonctionnalités transverses.

Une première tentative d'intégration avec un SIEM avait été testée, mais finalement retirée afin de maintenir un pipeline d'import stable, performant et facile à scaler.

3. Choix et justification des technologies

3.1. Backend : FastAPI + SQLAlchemy



Le backend repose sur **FastAPI**, un framework moderne et performant conçu pour les API web. Il a été choisi pour plusieurs raisons :

- **Performance élevée** grâce à Starlette et Uvicorn.
- **Simplicité de développement** avec une syntaxe claire et concise.
- **Validation automatique des données** via les modèles Pydantic.
- **Documentation interactive** générée automatiquement (Swagger/OpenAPI).



Pour l'accès aux données, l'application utilise **SQLAlchemy**, un ORM largement adopté dans l'écosystème Python. Son rôle principal est de gérer :

- La modélisation relationnelle (tables, relations),
- La création de sessions de base de données,
- La gestion des transactions,
- La compatibilité avec plusieurs moteurs SQL.

Cette combinaison permet d'obtenir une API structurée, testable et conforme aux bonnes pratiques du développement backend.

3.2. Base de données : PostgreSQL + SQLite

PostgreSQL (environnement de production)



PostgreSQL est le moteur principal utilisé pour le déploiement et les environnements sérieux. Il a été retenu pour :

- Sa **robustesse** et sa stabilité reconnue,
- Son excellent support des **agrégations**, essentielles pour le tableau de bord,

- Ses performances pour des volumes de données importants,
- Son écosystème riche (extensions, outils d'administration),
- Une gestion fiable de la **concurrence et des transactions**.

Le modèle représentant les URL, leurs domaines et leurs attributs est défini dans *models.py*, assurant une structure claire et extensible.

SQLite (développement local et tests)



Pour simplifier les phases de développement et de test, l'application peut fonctionner avec **SQLite**, un moteur léger ne nécessitant pas d'installation.

Il est utilisé pour :

- Les environnements isolés,
- L'exécution des tests unitaires (plus rapide),
- Les démonstrations ou scénarios hors réseau.

Cette flexibilité est rendue possible grâce à SQLAlchemy qui abstrait le moteur sous-jacent, sans changer le code métier.

3.3. Gestion du schéma : Alembic



La gestion des migrations de base de données est assurée par **Alembic**, un outil officiel associé à SQLAlchemy. Il permet :

- De versionner et d'historiser les modifications du schéma,
- De synchroniser automatiquement la base avec les modèles Python,
- De faciliter le déploiement sur des environnements cloud (AWS, Docker),
- De garantir une compatibilité entre les différentes versions de l'application.

Les migrations sont stockées dans le répertoire *migrations/* et doivent être exécutées lors de la mise en production ou lors d'une modification structurelle du modèle.

3.4. Frontend : Vue 3 + Vite



La partie client est développée en **Vue 3**, un framework JavaScript moderne, réactif et bien adapté aux interfaces dynamiques. Le choix est motivé par :

- Sa **courbe d'apprentissage raisonnable**,
- Sa logique de composants,
- Son excellente intégration avec TypeScript et Vite,
- Sa rapidité de rendu et de compilation.

Le bundler **Vite** permet un temps de compilation très court, ce qui améliore significativement la productivité durant le développement.

Les composants principaux incluent :

- **ImportCsv.vue** : import et envoi du fichier CSV vers l'API,
- **UrlTable.vue** : affichage, filtrage et gestion CRUD des URL,
- **dashboard.vue** : visualisation graphique des statistiques.

3.5. Cache : Redis



Certaines opérations, notamment :

- Les recherches filtrées,
- Les statistiques du tableau de bord,

Génèrent beaucoup de charge côté base de données.

Redis est ajouté comme **couche de cache** pour réduire la latence et améliorer les performances. Une petite abstraction interne (fichier cache.py) permet de centraliser facilement les opérations de lecture/écriture.

3.6. Conteneurisation : Docker + docker-compose



L'ensemble du projet est conteneurisé pour :

- Garantir un environnement constant entre machines,
- simplifier l'orchestration des services (backend, frontend, PostgreSQL, Redis),
- Préparer le déploiement vers **AWS**.

Les Dockerfiles sont construits en **multi-stage** afin de :

- Réduire la taille finale des images,
- accélérer les déploiements,
- Renforcer la sécurité (images plus compactes, surface d'attaque réduite).

3.7. Sécurité

Domaine	Technologie	Rôle Sécurité
Authentification	JWT + OAuth2	Identité, intégrité du token
Mots de passe	pbkdf2_sha256	Hashing sécurisé
Stockage secrets	dotenv	Aucune clé sensible n'est intégrée directement dans le code afin d'éviter toute exposition publique
Protection API	RBAC, dépendances FastAPI	Contrôle d'accès strict
Gestion JWT	HS256 + vérification manuelle exp	Token non falsifiable
Validation données	Pydantic	Anti-injection, entrées propres
Logs	JSON structuré	Monitoring, intégration SIEM
Conteneurisation	Docker	Isolation des services
Cache	Redis	Limiter les attaques "performance"
Frontend	axios interceptors, localStorage	Routes protégées

3.8. Autres outils

En complément des technologies principales, plusieurs outils ont été utilisés pour améliorer la qualité du développement, faciliter le déploiement et évaluer les performances du système.

Git & GitHub

Utilisés pour la gestion de version, la collaboration et le suivi des évolutions du projet. GitHub a également servi d'espace centralisé pour héberger le code, gérer les issues et assurer le versioning du backend, du frontend et de l'infrastructure Docker.

AWS CloudWatch

Outil de monitoring natif AWS utilisé pour suivre les métriques de performance de l'instance EC2 : charge CPU, trafic réseau, paquets traités.

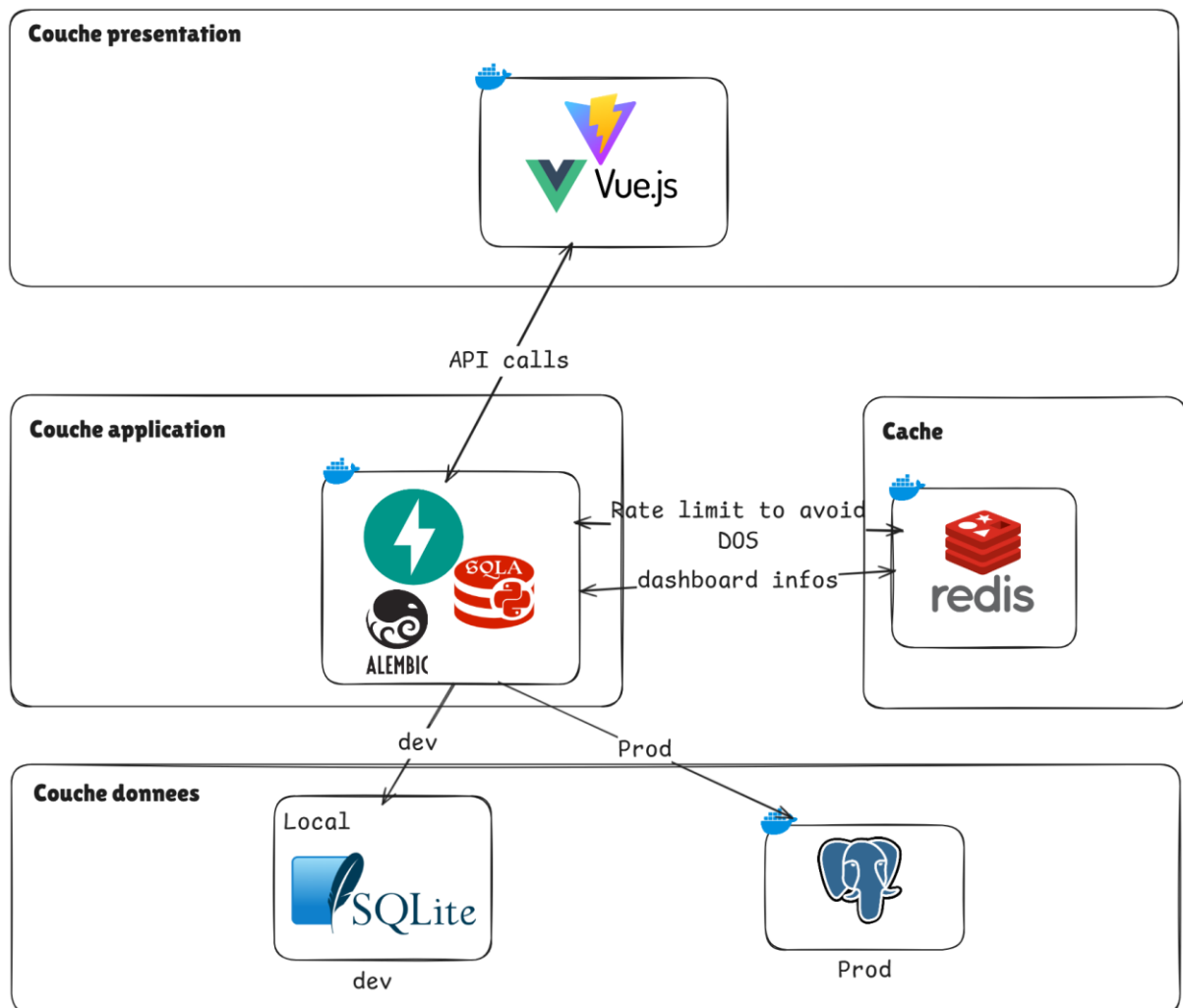
Il permet une supervision en temps réel et aide à valider la stabilité de l'infrastructure.

K6 — Outil de test de charge

K6 a été utilisé pour mesurer la capacité de l'application à supporter un trafic important et pour analyser son temps de réponse sous stress.

4. Mini-documentation de l'application

4.1. Architecture générale



L'application StopPhishingDotCom suit une architecture en couches classique pour les applications web, avec une séparation claire des responsabilités entre le front-end, le back-end et la persistance des données. On distingue trois couches principales :

1. Couche Présentation (Front-end)

- **Rôle** : Interface utilisateur pour visualiser, naviguer et interagir avec l'application.
- **Technologie** : Vue 3 avec Vite pour un rendu rapide et réactif.
- **Flux** : Toutes les interactions avec le serveur passent par des appels API.

2. Couche Application (Back-end et logique)

- **Rôle** : Cœur du système, gère la logique métier, l'authentification, le traitement des requêtes et l'accès aux données.
- **Composants** :
 - **Serveur Applicatif** : Traite les requêtes API et interagit avec la base de données via SQLAlchemy et Alembic pour les migrations.

- **Cache Redis :**
 - Limitation du nombre de requêtes pour prévenir les attaques DDoS.
 - Mise en cache des données du tableau de bord pour accélérer les réponses et réduire la charge sur la base principale.
- **Flux :** Communique avec la couche Présentation pour les API calls, avec la couche Données pour persister les informations et avec Redis pour performance et sécurité.

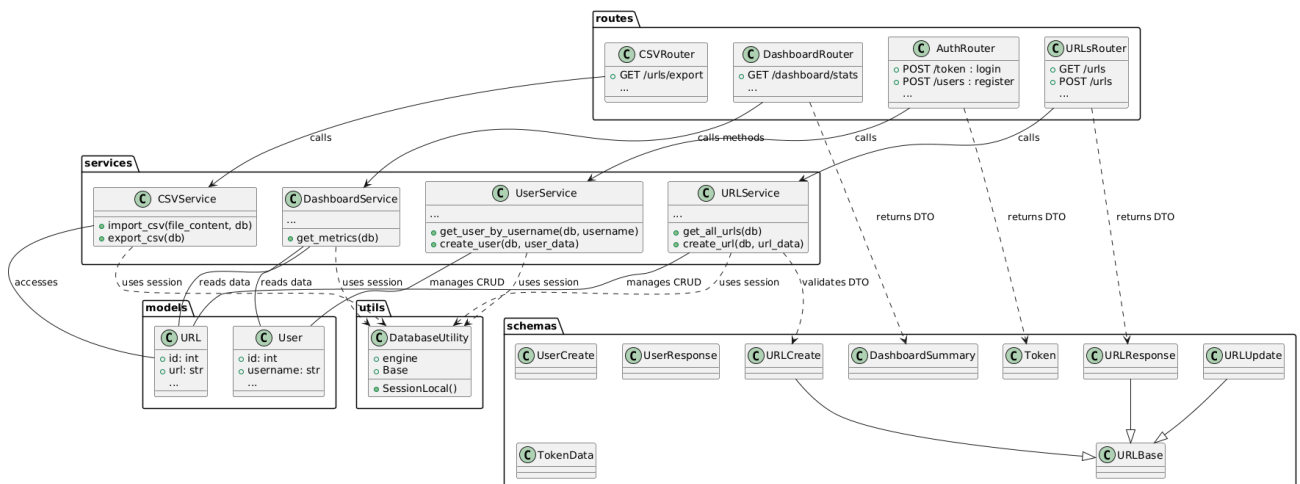
3. Couche Données (Persistance)

- **Rôle :** Stockage fiable et durable des informations.
- **Environnements :**
 - **Développement :** SQLite, simple et léger, adapté aux tests locaux.
 - **Production :** PostgreSQL, robuste et performant pour les volumes importants.
- **Flux :** La couche Application se connecte à la base adaptée selon l'environnement.

Synthèse :

- Front-end découplé du back-end.
- Back-end transactionnel et modulaire.
- Cache Redis pour performance et sécurité.
- Bases de données adaptées au contexte (SQLite dev, PostgreSQL prod).

4.2. Diagramme de classes



Le diagramme modélise les composants principaux et leur interaction selon la logique de séparation des responsabilités. L'application est structurée en cinq packages :

1. models (Persistance)

- Contient les modèles ORM (URL, User) qui correspondent aux tables de la base.

2. utils (Infrastructure)

- Fournit des utilitaires transversaux, notamment DatabaseUtility pour gérer le moteur, les sessions et la base ORM.

3. **schemas (DTOs / Transfert de données)**

- Définit les objets de transfert (URLCreate, Token, DashboardSummary) pour valider les entrées et formater les réponses, séparant les modèles métier de la base de données.

4. **services (Logique métier)**

- Contient la logique métier (URLService, UserService, DashboardService) : validation, calculs, orchestration des opérations CRUD et lecture de données.

5. **routes (API / Contrôleurs)**

- Définit les points de terminaison HTTP (URLsRouter, AuthRouter, DashboardRouter) et gère le cycle requête/réponse.

Flux d'exécution :

1. Un Router reçoit une requête HTTP.
2. Le Router appelle le Service correspondant pour appliquer la logique métier et valider les données via les schemas.
3. Le Service interagit avec la base via les modèles et DatabaseUtility pour persister ou lire les données.
4. Le Router formate la réponse avec un schema et renvoie au client.

Cas particuliers :

- **Authentification** : gérée par AuthRouter et UserService avec les schemas Token et UserCreate.
- **Tableau de bord** : DashboardRouter utilise DashboardService pour agréger les données URL et User en lecture seule.

Cette structure assure une maintenance simple, une extensibilité facile et une responsabilité claire pour chaque composant.

4.3. Mise en route (Docker)

1. Copier le fichier d'exemple :

```
cp .env.example .env
```

2. Lancer la stack :

```
docker compose up -d --build
```

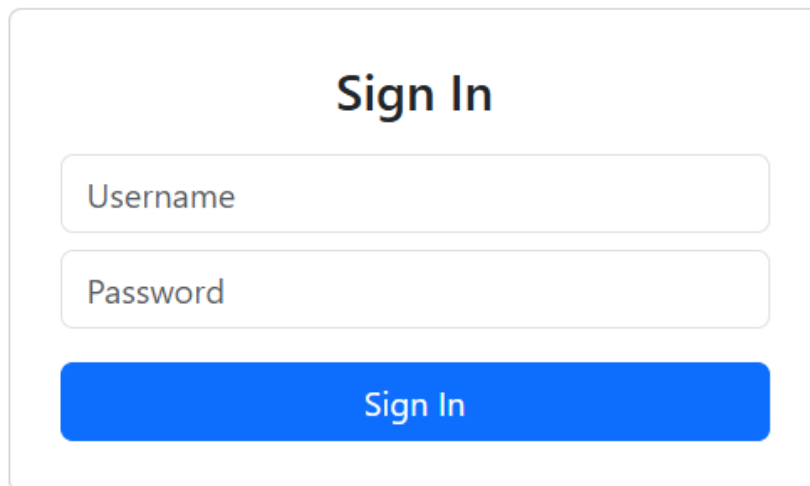
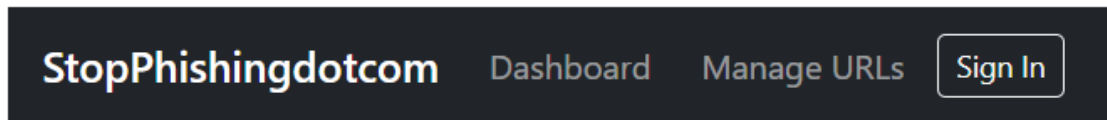
3. Appliquer les migrations :

```
docker compose exec backend sh -c "alembic upgrade head"
```

4.4. Implémentation :

L'implémentation de StopPhishingDotCom repose sur l'architecture décrite précédemment et se concentre sur trois pages principales : le **login**, le **dashboard** et la page **/urls**. Chaque page est reliée aux services backend via des appels API sécurisés et utilise les composants Vue pour l'affichage et l'interaction.

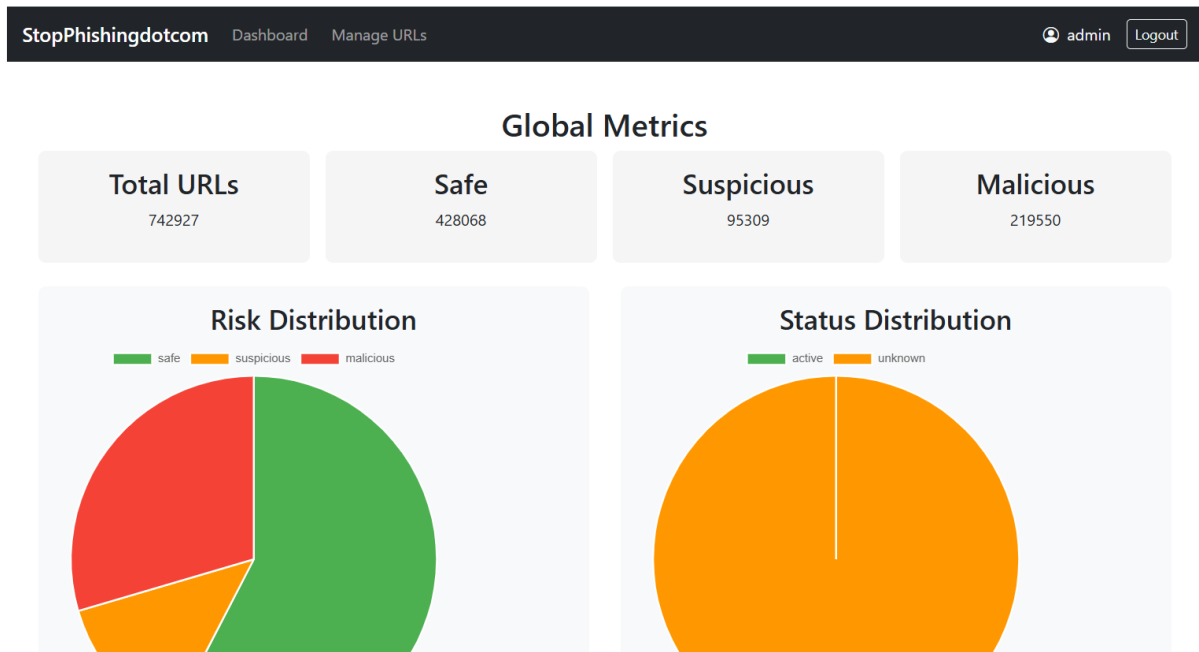
Page de connexion (Login)

A white rectangular form with rounded corners. At the top center is the title 'Sign In' in bold black text. Below the title are two input fields: the first is labeled 'Username' and the second is labeled 'Password'. Both labels are in a light gray font. Below the input fields is a large blue button with the text 'Sign In' in white.

- **Objectif** : Authentifier les utilisateurs et obtenir un token JWT pour accéder aux fonctionnalités protégées.
- **Fonctionnalités principales** :
 - Saisie de l'identifiant et du mot de passe.
 - Vérification des informations via l'API /token.
 - Stockage du token dans localStorage pour persistance de session.
 - Redirection vers le dashboard après authentification réussie.
- **Sécurité** : Les mots de passe sont hachés côté serveur avec pbkdf2_sha256 et les tokens JWT sont signés en HS256 avec expiration.
- **Flux** :
 1. L'utilisateur saisit ses identifiants.
 2. Le frontend envoie la requête POST à /token.
 3. Le backend valide les informations, renvoie un token JWT.

4. Le frontend stocke le token et permet l'accès aux routes protégées.

Page Dashboard



- **Objectif** : Visualiser les statistiques globales des URL importées.
- **Fonctionnalités principales** :
 - Graphiques interactifs (ligne, camembert) affichant le nombre d'URL par catégorie, par domaine ou par statut.
 - Requêtes aux endpoints /dashboard/stats pour récupérer les métriques.
 - Mise en cache des résultats avec Redis pour réduire la charge serveur.
- **Flux** :
 1. La page demande les métriques au DashboardService.
 2. Les données sont récupérées depuis PostgreSQL ou Redis (cache).
 3. Les graphiques sont rendus côté frontend avec Vue et mises à jour dynamiquement.

- **Objectif** : Gérer la liste complète des URL importées.
- **Fonctionnalités principales** :
 - Affichage paginé des URL avec recherche et filtrage.
 - Création, mise à jour et suppression d'URL (mise à jour et suppression réservée aux administrateurs).
 - Export et import CSV pour les opérations en masse.

- **Flux** :

1. Le frontend envoie les requêtes GET/POST/PUT/DELETE au service URLService via URLRouter.
2. Les données sont validées via les schémas Pydantic (URLCreate, URLUpdate).
3. Les opérations CRUD sont effectuées sur PostgreSQL avec gestion des transactions.
4. Les réponses sont formatées et renvoyées au client.

4.5. Déploiement :

Le projet peut être déployé sur des instances **EC2** :

Dans notre machine on se connecte avec notre instance via SSH (optionnel) :

```
C:\> ssh -i phishing.pem ubuntu@13.51.254.250
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1040-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Dec 7 20:00:21 UTC 2025
```

Dans la EC2 instance :

On installe docker et git

```
ubuntu@ip-172-31-5-21:~$ sudo apt update
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu/ focal InRelease
Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu/ focal-updates InRelease [119 kB]
Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu/ focal-backports InRelease [119 kB]
```

```
ubuntu@ip-172-31-5-21:~$ sudo apt install -y docker.io docker-compose
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz python3-docker python3-requests
ubuntu-fan
```

```
ubuntu@ip-172-31-5-21:~$ sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.34.1-1ubuntu1.15).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 27 not upgraded.
```

On clone le dépôt GitHub du projet.

```
ubuntu@ip-172-31-5-21:~$ git clone https://github.com/iman-coder/StopPhisingDotCom
Cloning into 'StopPhisingDotCom'...
remote: Enumerating objects: 309, done.
remote: Counting objects: 100% (227/227), done.
remote: Compressing objects: 100% (160/160), done.
remote: Total 309 (delta 108), reused 176 (delta 62), pack-reused 82 (from 1)
Receiving objects: 100% (309/309), 31.92 MiB | 24.03 MiB/s, done.
Resolving deltas: 100% (121/121), done.
```

On lance l'application avec Docker Compose

```
ubuntu@ip-172-31-5-21:~/StopPhisingDotCom$ docker-compose up -d --build
Creating network "stopphisingdotcom_default" with the default driver
8e44f01296e3: Pull complete
44fc29a25beb: Pull complete
39af693970c0: Pull complete
```

4.6. Tests

Le backend intègre une suite de tests unitaires pour garantir la fiabilité des services :

Exécuter les tests :

```
pytest -q
```

Vérifier l'état de la base de données en production :

Lister les tables existantes

```
exec -it stopphishing_db bash -c "psql -U stoppish -d stopphish"
5-1 pgdg13+1))
```

Les tables existantes :

```
stopphish=# \dt
          List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | alembic_version | table | stoppish
public | urls            | table | stoppish
public | users           | table | stoppish
(3 rows)
```

Tests de performance avec k6

Pour mesurer la performance des endpoints et simuler une charge d'utilisateurs, **k6** est utilisé. On peut lancer k6 directement via Docker sans l'installer localement :

1. Préparer le script de test (par exemple load_test.js) contenant la définition des endpoints et du scénario de charge.
2. Generation des csv pour tester les fonctionalites /import /export :

```
python tools/generate_csv.py 10000 test_10k.csv
```

3. Lancer le test avec Docker :

```
docker run --rm -i loadimpact/k6 run - <load_test.js
```

3. Si l'application tourne dans un réseau Docker Compose (ex. nom du service backend backend), exécuter :

```
docker run --rm -i --network="host" loadimpact/k6 run - <load_test.js
```

4. Analyse des résultats : k6 fournit des métriques clés telles que le **nombre d'itérations**, **latence moyenne**, **p(90)**, **p(95)**, **taux de réussite des requêtes** et **ressources consommées** (CPU, mémoire, réseau).

Ces tests permettent de vérifier la scalabilité et la résilience de l'application sous charge, en complément des tests unitaires fonctionnels.

5. Analyse de performance

5.1. Performance générale de l'application

Des tests de charge ont été réalisés afin d'évaluer la capacité de l'application à traiter un volume important de requêtes ainsi que la vitesse d'ingestion de fichiers CSV.

Les résultats montrent que le système est stable, performant et capable de traiter un flux soutenu avec un nombre limité d'utilisateurs simultanés.

5.2. Résultats des tests de charge (K6)

Les métriques suivantes ont été collectées avec **10 utilisateurs virtuels (VUs)** en parallèle :

Métrique	Valeur mesurée	Interprétation
VUs max	10	Le test a maintenu une charge constante.
Iterations	132 463	Nombre total de requêtes simulées.
Taux d'itérations	~2180 req/s	Indique que l'API supporte plus de 2000 requêtes par seconde dans ce scénario.
Durée moyenne	0.082 s	Réponse très rapide (<100 ms).
P90	0.088 s	90 % des requêtes répondent en < 90 ms.
P95	0.140 s	95 % des requêtes répondent en < 140 ms.
Durée max	17.9 s	Quelques pics liés au garbage collector ou au réseau, mais non représentatifs.

Analyse

- Les latences restent très faibles (0.08 s en moyenne), ce qui montre une bonne optimisation du backend FastAPI.
- Le système tient facilement **plus de 2000 opérations par seconde**, ce qui est excellent pour une application d'analyse.
- Les pics à ~17 s sont isolés et peuvent provenir du système hôte, du disque ou du scheduler Docker. Ils n'affectent pas la moyenne générale.

5.3. Performance de l'import CSV

Deux scénarios ont été mesurés, chacun sur un fichier de **10 000 URLs** :

Opération	Durée	Interprétation
Import initial (10k)	2.68	Très rapide grâce à la normalisation en mémoire + transaction

URLs)	sec	SQL groupée.
Réimport du même fichier	0.31 sec	Preuve que le système de déduplication est performant.

Analyse

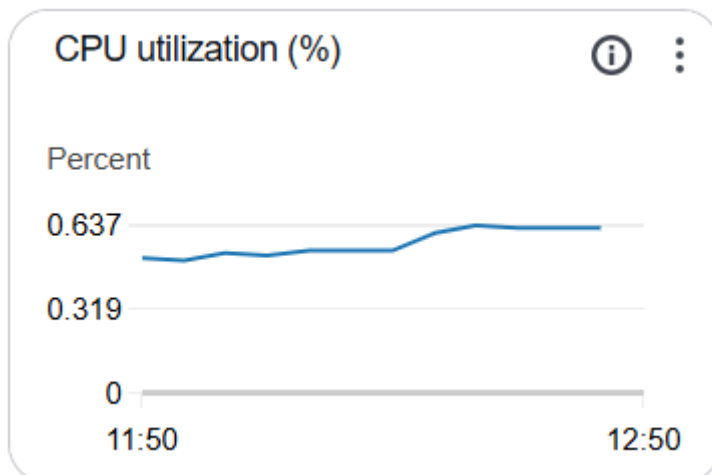
- Le pipeline d'import est capable d'insérer **≈ 3700 lignes/sec** lors du premier chargement.
- Lorsqu'un fichier déjà importé est renvoyé, le mécanisme de détection des doublons permet de réduire la durée de traitement de **≈ 88 %**.
- Le système peut donc traiter rapidement de gros volumes même en phase de nettoyage ou de recoupement.

Note : les tests k6 sont fait au localhost

5.4. Analyse de performance — Monitoring AWS

Un suivi en temps réel des performances a été réalisé via **Amazon CloudWatch** afin d'observer le comportement de l'instance EC2 hébergeant l'application. Les métriques analysées incluent l'utilisation CPU, le trafic réseau entrant/sortant et le nombre de paquets traités.

1. CPU Utilization



Les mesures montrent une charge CPU extrêmement faible :

- **Minimum : 0,61 %**
- **Maximum : 0,87 %**
- **Moyenne : 0,66 %**

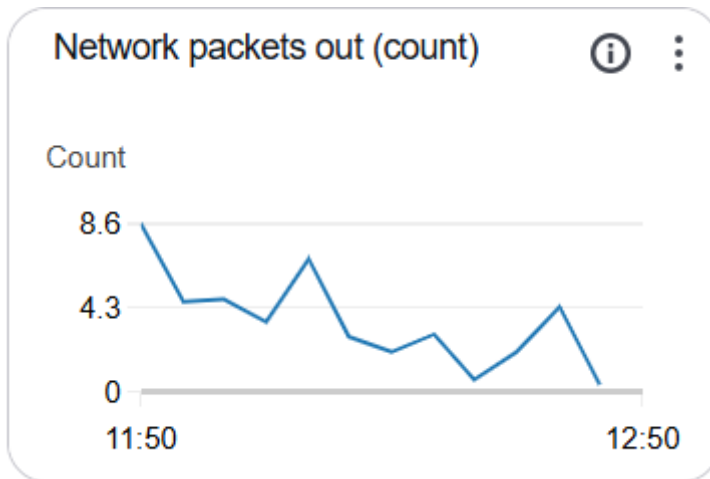
Interprétation

L'application consomme très peu de ressources CPU, même lors des périodes d'activité (import CSV, requêtes API).

Cela signifie que :

- FastAPI + PostgreSQL fonctionnent efficacement,
- L'instance EC2 est largement suffisante pour la charge actuelle,
- Il existe une marge importante pour absorber davantage de trafic sans mise à l'échelle immédiate.

Network Packets Out



Les paquets sortants varient selon les requêtes API et les communications internes :

- **Minimum : 0.4 packets/s**
- **Maximum : 10.8 packets/s**
- **Moyenne : 3.33 packets/s**

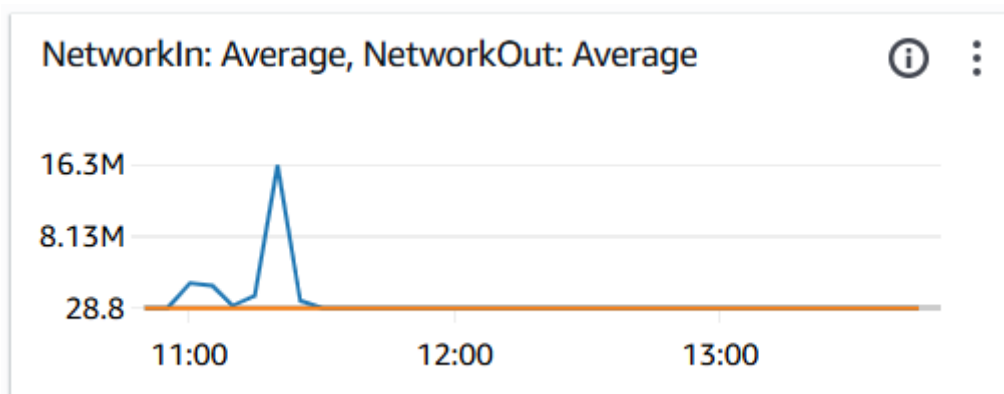
Interprétation

Le trafic sortant reste léger, cohérent avec une application principalement orientée API JSON. Les variations correspondent :

- Aux opérations CRUD sur les URLs,
- Aux rafraîchissements du tableau,
- Aux activités du dashboard.

Aucun pic anormal ne suggère une surcharge ou une fuite réseau.

NetworkIn & NetworkOut (volume de données)



NetworkIn

- **Minimum : 30 B**
- **Maximum : 16.3 MB**
- **Moyenne : 676 kB**
- **Somme totale mesurée : 24.3 MB**

NetworkOut

- **Minimum : 28.8 B**
- **Maximum : 92.6 kB**
- **Moyenne : 7.68 kB**
- **Somme totale mesurée : 276 kB**

Interprétation

Les volumes réseau sont cohérents avec :

- Des transferts de données modérés depuis le frontend (tableau, API),
- Des réponses principalement textuelles (JSON léger),
- Quelques pics liés aux imports CSV (lorsqu'un fichier transite vers le backend).

Ces chiffres montrent que :

- La bande passante utilisée est faible,
- Aucune saturation réseau n'est constatée,
- L'architecture actuelle supporte une montée en charge.

6. Conclusion et difficultés rencontrées

Le projet a permis de développer une solution complète et moderne pour l'import, l'analyse et la gestion d'URL malveillantes. L'application facilite la normalisation et la structuration des données, offrant aux analystes cybersécurité un outil efficace pour exploiter ces informations.

Principales difficultés rencontrées :

- Garantir la fiabilité lors des imports de fichiers CSV volumineux.
- Gestion de l'ordre de démarrage et des migrations entre conteneurs Docker.
- Ajustements nécessaires pour le build du front-end (versions de Node, dépendances).
- Intégration de l'authentification : mise en place des tokens JWT, gestion des rôles et protection des routes front-end.
- Conflits entre différentes API : certaines routes ou endpoints se chevauchaient, nécessitant une réorganisation des routers et des dépendances.
- Déploiement sécurisé en HTTPS : génération et configuration des certificats SSL/TLS pour la production sur AWS.
- Tentative initiale d'intégration SIEM en temps réel ayant provoqué des blocages, ce qui a conduit à sa suppression.
- Renforcer le mécanisme d'invalidation du cache pour garantir la cohérence des statistiques.

Enseignements :

Ce projet a permis de dégager plusieurs bonnes pratiques essentielles pour le développement d'applications web robustes et scalables :

- Maintenir une séparation claire des responsabilités entre les différentes couches (routes, services, utilitaires) pour faciliter la maintenance et l'extensibilité.
- Privilégier des opérations atomiques sur la base de données afin de garantir la cohérence des données et la sécurité des transactions.
- Surveiller et mesurer la performance dès les premières étapes du développement pour éviter les goulots d'étranglement et anticiper les optimisations nécessaires.
- Anticiper l'intégration future de fonctionnalités critiques (SIEM, traitement asynchrone, montée en charge sur le cloud) pour garantir une architecture flexible et évolutive.